Speculate in Order, Diversify in Disorder: A Rolling Shannon-Entropy Algorithm for Dynamic Top-10 Allocation

Algorithms Project with a Colab-Compatible Notebook

Databases and Algorithms Final Project

Abstract

This project solves a concrete quantitative-finance and algorithms problem: building a long-only stock portfolio every trading day over a dynamic top-10 equity universe, using only information available before the allocation date. The investment universe itself changes through time, so the algorithm must update both the active set of tradable stocks and the rolling statistics used to allocate capital. The strategy follows a simple principle: speculate when the market is ordered and diversify when it is disordered. To measure this ordered-to-disordered condition, the project uses rolling normalized Shannon entropy computed from recent stock returns. Entropy is not used as a direct return forecast. Instead, it classifies the market into Bullish, Neutral, or Bearish regimes, and the regime determines the maximum weight allowed for each active stock. The expected-return input remains the rolling mean, while the risk input is the rolling covariance matrix. The main algorithmic contribution is the rolling data-structure pipeline behind this signal: each stock is maintained through a rolling state, rolling minima and maxima are updated with monotonic queues, entropy is computed on adaptive return bins, and rolling entropy quantiles determine the daily regime without look-ahead bias. In Bullish and Neutral regimes, SLSQP solves the capped mean-variance allocation problem; in the Bearish regime, the cap forces the equal-weight top-10 portfolio and the optimizer is skipped. The final implementation treats the CSV files as already prepared inputs and compares the entropy-regime portfolio with the dynamic equal-weight top-10 benchmark and the S&P 500 benchmark.

Contents

1 Introduction

1.1 The Real Problem

The goal of this project is concrete and practical: build a stock portfolio. More precisely:

> At each trading date, choose a feasible long-only allocation across the current top-10 equity universe, using only historical information available before that date, while keeping every company outside the current top-10 at zero weight.

This is a genuine and non-trivial problem because the opportunity set is not fixed. The largest companies in the index change as market capitalizations move, firms enter or leave the index, and corporate events affect prices and shares outstanding. A strategy that assumes the same tickers forever is easier to implement, but it is less realistic than a dynamic-universe allocation rule. Everything else in the project, including entropy, regimes, and optimization, is machinery introduced to solve this allocation problem.

1.2 Why We Work on the Time Series

Building a portfolio over N assets can be approached from several angles. A fundamental approach selects companies from balance-sheet data such as earnings, debt, margins, and valuation ratios. An economic approach starts from macroeconomic variables such as inflation, unemployment, interest rates, and the business cycle. A mathematical/statistical approach works directly on the time series of past returns, without opening a balance sheet or a macro model. This project takes the third route: it uses only the return series of the investable universe.

A return series would be memoryless if the distribution of the return at time $t+1$ did not depend on the past. If that were true, one static distribution would describe the data and one could allocate from one fixed estimate. Financial returns do not behave this way: volatility and higher moments visibly depend on the recent past.

A weaker assumption is simple Markovian stationarity, where the distribution at $t+1$ depends only on the state at t. Under this assumption one could work with recent data and relatively simple updates. Real return series, however, display non-stationarity, volatility clustering, regime changes, and often long-range dependence, so a single fixed state is not enough either.

Equity markets behave differently in different contexts. Calm trending phases, choppy sideways phases, and turbulent sell-offs are statistically distinct. It is therefore natural to identify the current phase of the market and adapt the allocation rule.

1.3 The Strategy

The strategy is deliberately simple: concentrate capital when the market is ordered and spread capital toward equal weight when it is disordered. Entropy is the tool used to measure this order-disorder condition, not the final objective of the portfolio. When recent price behavior is structured and calm, a few names may carry a usable signal, so the portfolio can concentrate and try to extract return. When recent behavior is turbulent and disordered, no name is trustworthy, so the portfolio should stop trying to pick winners and spread out toward an equal-weight book that simply survives.

This is not an on/off switch. The algorithm reads a spectrum through three regimes:

$$\begin{aligned} &\text{Bullish} : \text{ordered market, concentration allowed,} \\ &\text{Neutral} : \text{balanced regime,} \\ &\text{Bearish} : \text{disordered market, diversification to equal weight.} \end{aligned}$$

The whole approach hinges on one measurement: how ordered or disordered is the market right now? Answering that question day by day and without peeking at the future is the purpose of the rolling entropy signal.

1.4 State of the Art and Motivation

Portfolio-allocation algorithms often rely on three broad families of methods:

1. Mean-variance optimization, where expected returns and covariances are estimated from historical data and a constrained quadratic problem is solved.
2. Momentum and trend filters, where recent average returns, rankings, or moving averages determine which assets appear attractive.
3. Regime or uncertainty indicators, where volatility, drawdown, clustering methods, kernel methods, Markov-switching models, hidden Markov models (HMMs), or entropy decide whether recent patterns are stable enough to trust.

The classical mean-variance framework is transparent and mathematically tractable, but it is sensitive to estimation error, especially in expected returns. For this reason, practical implementations often add long-only constraints, maximum-weight bounds, covariance shrinkage, regularization, or other stabilizing devices.

Momentum-based methods are simple and computationally efficient, but they can suffer when market leadership changes quickly or when recent winners reverse.

Regime-aware methods address a different question: they ask whether the current market environment is stable enough to apply an aggressive allocation rule. Equity markets behave differently in different contexts: calm trending phases, choppy sideways phases, and turbulent sell-offs are statistically distinct. It is therefore natural to identify the current phase of the market and adapt the allocation rule. This point is central for the present project, because the strategy does not want to concentrate capital with the same intensity in every market condition.

This project combines these ideas. Mean-variance optimization is used as the allocation engine. Rolling means and rolling covariances provide the standard return and risk inputs. Entropy provides the regime signal that controls the level of concentration.

The motivation is therefore not to replace classical portfolio optimization, but to make it conditional on the current structure of the market. When the market appears more ordered, the allocation rule is allowed to be more speculative. When the market appears more disordered, the rule becomes more defensive and moves toward diversification. In this sense, entropy is used as a market-state filter placed before the allocation step: it determines how restrictive the portfolio constraints should be, while the optimizer still determines the weights inside those constraints.

1.5 Algorithmic Contribution

The main contribution of the project is the online algorithmic pipeline, where "online" means that the algorithm processes the data sequentially through time: when a new trading date arrives, the rolling structures are updated using only past information, without recomputing the entire history from scratch and without using future observations. This pipeline turns a changing equity universe and rolling return data into daily portfolio weights. The algorithm must update information through time, respect causality, classify the market regime, and produce a feasible allocation at each trading date.

The custom part of the project is the rolling market-filter stack. Each stock is maintained through a rolling state that stores the most recent returns and the sufficient information needed for fast updates. Rolling minima and maxima are updated through monotonic queues, so the adaptive range used for entropy does not need to be recomputed from scratch at every date. The entropy routine then builds adaptive bins on the current rolling range and computes normalized Shannon entropy. Finally, a rolling quantile structure stores the active-universe entropy history and provides the thresholds used to classify the current regime.

The continuous optimizer is not the algorithmic focus of the project. It is a standard, replaceable numerical component used after the custom rolling signal has been built. In particular, SLSQP is used only to solve the capped mean-variance problem in Bullish and Neutral regimes, while the Bearish regime is handled directly through the equal-weight allocation. The core algorithmic work is the online construction of the regime signal and its integration into a feasible dynamic-universe allocation rule.

The final workflow provides:

1. a dynamic top-10 universe read from prepared CSV files;
2. daily price-index inputs suitable for return computation;
3. one rolling state per stock;
4. monotonic queues for rolling minima and maxima;
5. adaptive-bin normalized Shannon entropy;
6. rolling entropy quantiles for regime classification;
7. a regime-dependent maximum-weight cap;
8. SLSQP optimization in Bullish and Neutral regimes;
9. explicit equal weighting in the Bearish regime;
10. a head-to-head comparison against the dynamic equal-weight top-10 and the S&P 500 benchmark.

The dataset-building code is intentionally excluded from the Colab notebook. The CSV files are treated as already prepared inputs derived locally from `Equity_Data_Eikon.h5`. The provided `dataset.csv` contains the S&P 500 benchmark and 36 stock price-index series, runs from 2001-01-31 through 2024-12-31, and uses no continuation beyond 2024. All numerical tables in this report are computed exclusively from these prepared CSV files. For replicability, the References section will include a GitHub repository where the prepared datasets can be downloaded.

1.6 Why a Top-10 Universe and Why Entropy

The choice of a top-10 universe follows from the strategy. Once a phase is known, the strategy allocates among the companies with the largest current capitalizations. The top-10 are therefore large-cap market leaders, not a separate momentum signal. Recalculating the active list every trading day keeps the investable set aligned with the market; a company becomes tradable only once it genuinely enters the daily top-10 universe.

The choice of entropy is an example of econophysics: concepts from natural sciences are adapted to finance. Shannon entropy, originally an information-theoretic measure of disorder, becomes here an operational regime signal that drives a constrained allocation problem.

In this project, entropy is therefore not a forecasting model but a control variable for portfolio aggressiveness. The algorithm asks whether recent market behavior is ordered enough to justify concentration or disordered enough to require diversification. The answer determines the constraint level, and the constrained allocation rule then produces the final portfolio weights.

2 Modeling

The previous section introduced the economic intuition of the strategy. This section builds the mathematical model behind that intuition. The goal is to transform the real portfolio problem into a formal input-output problem that can be solved by an automated algorithm.

The model receives cleaned price data, a daily top-10 universe, and a fixed set of hyperparameters. It then constructs rolling return statistics, converts entropy into a market phase, and uses that phase to define a constrained long-only allocation problem. The output of the model is the portfolio generated by the allocation rule: the daily weight vector, the realized portfolio return series, and the performance statistics computed from it, such as annualized return, Sharpe ratio, maximum drawdown, and final wealth.

2.1 Model Inputs and Hyperparameters

Let $t = 1, \ldots, T$ index the trading dates in the sample, and let $i = 1, \ldots, N$ index the stock series contained in the prepared dataset. The model starts from cleaned CSV files, not from the raw Eikon database. The raw cleaning stage was performed separately before the implementation phase. In the notebook, `dataset.csv` provides the price panel, while `top10_dictionary.csv` provides the daily top-10 universe.

Let $\mathcal{U}_t$ denote the active universe at date t. It is the set of stock indices selected by the daily top-10 file:

$$\mathcal{U}_t \subseteq \{1, \ldots, N\}.$$

Since the strategy is defined on the current top-10 companies, the allocation problem is defined on an active universe of fixed size

$$K = |\mathcal{U}_t| = 10.$$

The set $\mathcal{U}_t$ changes through time, but its size is fixed. The mathematical problem is therefore dynamic because the admissible assets change with t, while the allocation rule remains the same.

The model hyperparameters are fixed before the simulation:

$$W = 126, \qquad B = 8, \qquad Q = 504, \qquad \lambda = 4.$$

The parameter W is the length of the rolling return window. The model needs this window to observe the recent dispersion of each stock's return time series and to compute rolling quantities such as entropy, mean, variance, and covariance.

The parameter B is the number of bins used to discretize the return window before computing Shannon entropy. In this project, each stock's rolling return window is divided into $B = 8$ uniformly spaced bins between its own rolling minimum and rolling maximum. This makes the entropy computation adaptive to each stock's recent range.

The parameter Q is the rolling entropy-quantile window. It is used after the stock-level entropies have been averaged across the active universe. The recent history of $\bar{H}_t$ over Q observations provides the 33rd and 67th percentile thresholds used to identify the market phase.

The parameter λ is the risk-aversion coefficient in the mean-variance objective. It controls the trade-off between expected return and portfolio risk once the entropy regime has determined the relevant maximum weight.

The set of possible entropy regimes is fixed as

$$\mathcal{S} = \{\text{Bullish}, \text{Neutral}, \text{Bearish}\}.$$

The regime-dependent concentration caps are also fixed in advance:

$$c(\text{Bullish}) = 0.30, \qquad c(\text{Neutral}) = 0.15, \qquad c(\text{Bearish}) = 0.10.$$

These parameters are not estimated by searching for the best historical performance. They are part of the model specification.

2.2 Return Series and Causal Information Set

Let $P_{i,t}$ be the adjusted price index of stock i on trading day t. The daily simple return is

$$r_{i,t} = \frac{P_{i,t}}{P_{i,t-1}} - 1.$$

At date t, the model can use only information available before $r_{i,t}$ is realized. Therefore, all rolling inputs for the date-t allocation are computed from the trailing window

$$R_{i,t}^{(W)} = (r_{i,t-W}, \ldots, r_{i,t-1}).$$

The realized return $r_{i,t}$ is used only after the portfolio has been chosen, to compute the date-t portfolio return.

2.3 Rolling Entropy Signal

Entropy is used to measure how ordered or disordered recent returns are. Since different stocks can have different volatility levels, entropy is computed on an adaptive range for each stock and each date.

For stock i at date t, define the rolling minimum and rolling maximum as

$$m_{i,t} = \min_{\tau=t-W,\ldots,t-1} r_{i,\tau},$$

and

$$M_{i,t} = \max_{\tau=t-W,\ldots,t-1} r_{i,\tau}.$$

The interval $[m_{i,t}, M_{i,t}]$ is divided into B bins. Let $n_{b,i,t}$ be the number of observations of stock i falling into bin b at date t. The empirical bin probability is

$$p_{b,i,t} = \frac{n_{b,i,t}}{W}.$$

The normalized Shannon entropy of stock i is

$$H_{i,t} = -\frac{1}{\log_2 B} \sum_{b=1}^{B} p_{b,i,t} \log_2(p_{b,i,t}),$$

where empty bins contribute zero. The normalization maps entropy to $[0, 1]$. If all observations fall into one bin, entropy is 0. If observations are spread evenly across the bins, entropy is 1.

The active-universe entropy is the average entropy across the current top-10 stocks:

$$\bar{H}_t = \frac{1}{K} \sum_{i \in \mathcal{U}_t} H_{i,t}.$$

This is the model's daily disorder signal. Lower values of $\bar{H}_t$ indicate that recent returns are more concentrated in a few empirical states. Higher values indicate that recent returns are more spread across the available states.

2.4 Entropy Regime Classification

The entropy signal is classified using rolling quantiles rather than fixed numerical thresholds. The lower and upper entropy thresholds are

$$q_{0.33,t} = \text{Quantile}_{0.33}\left(\bar{H}_{t-Q}, \ldots, \bar{H}_{t-1}\right),$$

and

$$q_{0.67,t} = \text{Quantile}_{0.67}\left(\bar{H}_{t-Q}, \ldots, \bar{H}_{t-1}\right).$$

Only past active-universe entropy observations are used in these thresholds.

The market phase at date t is defined as

$$\text{phase}_t = \begin{cases} \text{Bullish}, & \bar{H}_t \le q_{0.33,t}, \\ \text{Bearish}, & \bar{H}_t \ge q_{0.67,t}, \\ \text{Neutral}, & \text{otherwise.} \end{cases}$$

The interpretation is:

$$\begin{aligned} &\text{Bullish : ordered market, concentration allowed,} \\ &\text{Neutral : intermediate market state,} \\ &\text{Bearish : disordered market, diversification enforced.} \end{aligned}$$

Entropy therefore does not forecast returns directly. It only determines the regime and, through the regime, the concentration constraint imposed on the optimizer.

2.5 Expected Return and Risk Inputs

The expected-return input is the plain rolling mean of past returns:

$$\widehat{\mu}_{i,t} = \frac{1}{W}\left(r_{i,t-W} + \sum_{\tau=t-W+1}^{t-2} r_{i,\tau} + r_{i,t-1}\right).$$

The term $r_{i,t-W}$ is the oldest observation in the window, while $r_{i,t-1}$ is the most recent return available before the date-t decision. The summation term contains the central part of the rolling window.

The rolling variance is computed on the same causal window:

$$\widehat{\sigma}^2_{i,t} = \frac{1}{W-1}\left[(r_{i,t-W} - \widehat{\mu}_{i,t})^2 + \sum_{\tau=t-W+1}^{t-2} (r_{i,\tau} - \widehat{\mu}_{i,t})^2 + (r_{i,t-1} - \widehat{\mu}_{i,t})^2\right].$$

For two active stocks $i, j \in \mathcal{U}_t$, the rolling covariance is

$$\widehat{\Sigma}_{ij,t} = \frac{1}{W-1}\sum_{\tau=t-W}^{t-1} (r_{i,\tau} - \widehat{\mu}_{i,t})\,(r_{j,\tau} - \widehat{\mu}_{j,t}).$$

The active covariance matrix is

$$\Sigma_t = \left(\widehat{\Sigma}_{ij,t}\right)_{i,j\in\mathcal{U}_t}.$$

Thus, the model separates the roles of the main inputs: the rolling mean gives direction, the covariance matrix gives risk, and entropy gives the concentration regime.

2.6 Portfolio Optimization Model

Let $w_{i,t}$ be the portfolio weight assigned to stock i at date t. The active weight vector is

$$w_t = (w_{i,t})_{i\in\mathcal{U}_t}.$$

Stocks outside the active universe are not admissible in the date-t allocation problem:

$$w_{i,t} = 0 \qquad \text{for all } i \notin \mathcal{U}_t.$$

Let

$$\widehat{\mu}_t = (\widehat{\mu}_{i,t})_{i\in\mathcal{U}_t}$$

be the rolling expected-return vector of the active stocks. In Bullish and Neutral phases, the allocation problem is the long-only mean-variance problem

$$\min_{w_t} \frac{\lambda}{2} w_t^\top \Sigma_t w_t - \widehat{\mu}_t^\top w_t$$

subject to

$$\sum_{i \in \mathcal{U}_t} w_{i,t} = 1,$$

and

$$0 \le w_{i,t} \le c(\text{phase}_t) \qquad \text{for all } i \in \mathcal{U}_t.$$

In Bearish phases, the model does not require a numerical optimizer. Since the active universe has ten stocks and the Bearish cap is 10%, the full-investment constraint forces

$$w_{i,t} = 0.10 \qquad \text{for all } i \in \mathcal{U}_t.$$

Therefore, the Bearish phase corresponds exactly to the equal-weight top-10 portfolio.

2.7 Model Outputs

The primary output of the model is the daily strategy return. Once the weights have been chosen, the date-t realized return is compared with two benchmark returns:

$$R_t^{\text{strat}} = \sum_{i \in \mathcal{U}_t} w_{i,t} r_{i,t}, \qquad R_t^{\text{EW}} = \frac{1}{K} \sum_{i \in \mathcal{U}_t} r_{i,t}, \qquad R_t^{\text{SP500}}.$$

The first series is the entropy-regime portfolio return. The second is the dynamic equal-weight top-10 return, computed on the same active universe as the strategy. The third is the S&P 500 benchmark return, which is external to the optimization universe and is used only for performance comparison.

The complete model output is

$$\left(R_t^{\text{strat}},\, R_t^{\text{EW}},\, R_t^{\text{SP500}},\, w_t,\, \bar{H}_t,\, \text{phase}_t\right)_t.$$

These objects are then used in the simulation section to compute performance statistics, phase counts, transition probabilities, crisis-window returns, solver diagnostics, and weight-bound diagnostics.

2.8 Performance Statistics

Let $\mathcal{T}$ be the set of allocation dates and let $|\mathcal{T}|$ be the number of realized portfolio returns. For a generic return series R_t, the cumulative wealth process is

$$C_t = \prod_{\tau \le t} (1 + R_\tau), \qquad C_0 = 1.$$

The same formulas are applied to the entropy strategy, the dynamic equal-weight top-10 benchmark, and the S&P 500 benchmark.

The main performance measures are annualized return, annualized volatility, Sharpe ratio, maximum drawdown, and final wealth:

$$R_{\text{ann}} = C_T^{252/|\mathcal{T}|} - 1, \qquad \sigma_{\text{ann}} = \sqrt{252}\ \text{Std}(R_t), \qquad \text{Sharpe} = \frac{R_{\text{ann}}}{\sigma_{\text{ann}}}, \qquad C_T = \text{final wealth}.$$

The risk-free rate is set to zero, so the Sharpe ratio is computed directly as annualized return divided by annualized volatility.

Maximum drawdown measures the largest peak-to-trough loss of the cumulative wealth curve:

$$C_t^{\max} = \max_{\tau \le t} C_\tau, \qquad DD_t = \frac{C_t - C_t^{\max}}{C_t^{\max}}, \qquad \text{MaxDD} = \min_t DD_t.$$

The phase counts and phase shares summarize how often the algorithm is in each entropy regime:

$$N_s = \sum_{t \in \mathcal{T}} \mathbf{1}\{\text{phase}_t = s\}, \qquad \pi_s = \frac{N_s}{|\mathcal{T}|}, \qquad s \in \{\text{Bullish}, \text{Neutral}, \text{Bearish}\}.$$

Transition probabilities measure how frequently the model moves from one phase to another:

$$\widehat{P}_{ab} = \frac{\sum_t \mathbf{1}\{\text{phase}_t = a,\ \text{phase}_{t+1} = b\}}{\sum_t \mathbf{1}\{\text{phase}_t = a\}}, \qquad a, b \in \mathcal{S}.$$

They are used to check whether the entropy regimes are persistent or unstable.

For a crisis window $\mathcal{C} = [t_0, t_1]$, the cumulative return is

$$R_{\mathcal{C}} = \prod_{t \in \mathcal{C}} (1 + R_t) - 1.$$

This formula is applied separately to the strategy, the equal-weight top-10 benchmark, and the S&P 500 benchmark.

Finally, solver diagnostics are recorded to check that the numerical optimization step behaves correctly. They include solver success, objective value, number of iterations, and feasibility of the portfolio constraints:

$$\sum_{i \in \mathcal{U}_t} w_{i,t} = 1, \qquad 0 \le w_{i,t} \le c(\text{phase}_t) \quad \text{for all } i \in \mathcal{U}_t.$$

Weight-bound diagnostics also report the minimum and maximum active weights and how often the optimizer places stocks at the lower or upper bound.

2.9 Causality and Look-Ahead Bias

The model avoids look-ahead bias because every date-t decision uses only information available before the date-t return is realized. The return window ends at $t-1$, the entropy thresholds are computed from past entropy observations, and the active universe is selected with information observed no later than $t-1$.

This causal ordering can be summarized as

$$\Big(\mathcal{U}_t,\ R_{i,t}^{(W)},\ \bar{H}_t,\ q_{0.33,t},\ q_{0.67,t},\ \widehat{\mu}_t,\ \Sigma_t\Big) \longrightarrow w_t \longrightarrow R_t^{\text{strat}}.$$

The portfolio weights are therefore computed before the realized portfolio return is observed. This ordering is essential: without it, the backtest would use future information and the reported performance would not represent an implementable strategy.

3 Algorithm Presentation

This section explains how the mathematical model is turned into an automated procedure. The model defines the quantities that must be computed, while the algorithm specifies the order in which these quantities are produced and used to obtain the daily portfolio.

The strategy works as a daily pipeline. First, it loads the prepared inputs and maintains rolling information for every stock. At each trading date, it reads the current top-10 universe, computes entropy-based signals from past returns, identifies the market phase, builds the portfolio weights, and finally evaluates the realized performance of the strategy against reference portfolios.

The first feature to notice is the rolling-window structure of the problem. The strategy uses a window of length $W = 126$ for stock-level returns, rolling mean, variance, covariance, minimum, maximum, and entropy. A second window of length $Q = 504$ is used for the time series of active-universe average entropy, which is needed to compute the regime thresholds.

The full procedure can be divided into smaller algorithmic subproblems. Each subproblem solves one specific part of the strategy and passes its output to the next step. This section does not develop the data structures in detail: that discussion is left to the Algorithm Analysis section, where deques, monotonic queues, sorted lists, rolling buffers, and computational complexity are analyzed. Here, we only present the main pseudocode blocks needed to solve the modeled problem: Shannon entropy, rolling entropy quantiles, daily regime-driven portfolio construction, and final performance comparison.

3.1 Pseudocode for Shannon Entropy

The entropy routine receives one rolling window of returns, the rolling minimum, the rolling maximum, and a number of bins. It returns a normalized entropy value in $[0, 1]$. Low entropy means that returns are concentrated in a few empirical states; high entropy means that returns are more spread across the bins.

In the implementation, the rolling minimum and maximum are not recomputed inside the entropy routine. They are provided by the rolling stock state, which maintains them through dedicated monotonic-queue structures. This reduces the computational cost because the entropy function receives the current lower and upper bounds directly.

Algorithm 1 ShannonEntropy: normalized entropy of one rolling return window

Require: return window $x = (x_1, \ldots, x_W)$; rolling minimum lo; rolling maximum hi; bin count $B \geq 2$
Ensure: normalized entropy $H_{\text{norm}} \in [0, 1]$

if $hi = lo$ **then**
  return 0 ▷ degenerate window: all returns equal
end if
$\Delta \leftarrow (hi - lo)/B$ ▷ equal-width bin size
$c \leftarrow (0, \ldots, 0)$ ▷ array of B bin counts
for each value v in x **do**
  $b \leftarrow \min\left(\lfloor (v - lo)/\Delta \rfloor, B - 1\right)$
  $c_b \leftarrow c_b + 1$
end for
$\mathcal{P} \leftarrow \{c_b/W : c_b > 0\}$ ▷ empty bins skipped
$H \leftarrow -\sum_{p \in \mathcal{P}} p \log_2 p$
return $H / \log_2 B$ ▷ normalization to $[0, 1]$

Read from top to bottom, the routine has three stages. First, it receives the current adaptive range $[lo, hi]$ from the rolling stock state. Second, it builds a histogram over B equal-width bins. Third, it converts the non-empty bin counts into probabilities and computes normalized Shannon entropy. The range is window-adaptive because each stock is evaluated using its own recent return range, but the range is maintained outside the entropy routine for efficiency.

3.2 Pseudocode for Rolling Entropy Quantiles

After the active-universe average entropy $\bar{H}_t$ has been computed through time, the algorithm needs two rolling thresholds: the 33rd percentile and the 67th percentile of the most recent Q average-entropy observations. These thresholds split the entropy history into three regions and are used to classify the market phase as Bullish, Neutral, or Bearish.

The rolling quantile state is defined as $Q_{\text{state}} = (\texttt{buffer}, \texttt{sorted\_values})$. The first component, `buffer`, stores the most recent average-entropy values in chronological order. It is used to identify which value is the oldest when the rolling window moves forward. The second component, `sorted_values`, stores the same values in increasing order. It is used to compute the 33rd and 67th percentiles by rank.

Algorithm 2 RollingEntropyQuantiles: compute interpolated thresholds and update the window

Require: rolling quantile state $Q_{\text{state}} = (\texttt{buffer}, \texttt{sorted\_values})$; new average entropy $\bar{H}_t$; window length Q
Ensure: lower threshold q_{lo}, upper threshold q_{hi}, and updated quantile state

if $|Q_{\text{state}}.\text{buffer}| < Q$ **then**
  $q_{lo} \leftarrow$ NaN
  $q_{hi} \leftarrow$ NaN ▷ not enough past entropy observations
else
  $s \leftarrow Q_{\text{state}}.\text{sorted\_values}$ ▷ last Q entropy values in increasing order
  $q_{lo} \leftarrow \text{InterpolatedQuantile}(s, 0.33)$
  $q_{hi} \leftarrow \text{InterpolatedQuantile}(s, 0.67)$
end if
if $|Q_{\text{state}}.\text{buffer}| = Q$ **then**
  $h_{\text{old}} \leftarrow Q_{\text{state}}.\text{buffer.popleft()}$ ▷ remove oldest value in chronological order
  remove h_{old} from $Q_{\text{state}}.\text{sorted\_values}$ ▷ remove the same value from sorted order
end if
$Q_{\text{state}}.\text{buffer.append}(\bar{H}_t)$ ▷ insert new value in chronological order
insert $\bar{H}_t$ into $Q_{\text{state}}.\text{sorted\_values}$ in the correct sorted position ▷ preserve increasing order
return (q_{lo}, q_{hi})

The first part of the routine reads the two thresholds from the values already stored in Q_{state}. Since the values are kept in sorted order, the algorithm can locate the positions corresponding to the 33rd and 67th percentiles. When the desired position falls between two sorted values, the quantile is computed by linear interpolation:

$$q_\alpha = (1 - \omega)s_\ell + \omega s_u.$$

Here, s_ℓ and s_u are the two adjacent sorted entropy values, and ω is the interpolation weight between them. The same rule is used for both q_{lo} and q_{hi}.

The second part updates the state. If the rolling window is already full, the oldest entropy value is removed from both `buffer` and `sorted_values`. Then the new average entropy $\bar{H}_t$ is inserted into both structures. After the update, the two structures contain the same entropy values: one in chronological order and one in increasing order.

3.3 Pseudocode for the Portfolio Algorithm

At this stage, the rolling quantities required for the portfolio decision are already available: the current active universe, the active-universe average entropy, the two entropy thresholds, the rolling expected returns, and the covariance matrix. Therefore, this subproblem only identifies the market phase and translates it into portfolio weights.

Algorithm 3 Daily regime-driven portfolio construction

Require: daily top-10 universe $\mathcal{U}_t$; average entropy $\bar{H}_t$; entropy thresholds $q_{lo,t}$ and $q_{hi,t}$; expected returns $\widehat{\mu}_t$; covariance matrix Σ_t; risk aversion $\lambda = 4$
Ensure: daily strategy weights w_t and market phase phase_t

for each trading date t in chronological order **do**
  if all required rolling inputs are available for date t **then**
    if $\bar{H}_t \le q_{lo,t}$ **then**
      $\text{phase}_t \leftarrow$ Bullish
    else if $\bar{H}_t \ge q_{hi,t}$ **then**
      $\text{phase}_t \leftarrow$ Bearish
    else
      $\text{phase}_t \leftarrow$ Neutral
    end if
    if $\text{phase}_t =$ Bearish **then**
      $w_t \leftarrow (1/10, \ldots, 1/10)$ ▷ uniform allocation over the active top-10 universe
    else
      $c_t \leftarrow 0.30$ if Bullish, else 0.15
      $w_t \leftarrow \text{SLSQP}\left(\frac{\lambda}{2} w^\top \Sigma_t w - \widehat{\mu}_t^\top w\right)$ subject to $\sum_{i \in \mathcal{U}_t} w_i = 1$ and $0 \le w_i \le c_t$
    end if
    save w_t and phase_t
  end if
end for

The output of this subproblem is the time series of strategy weights. The Bearish phase uses a uniform allocation over the active top-10 universe. The Bullish and Neutral phases solve the same mean-variance objective, but with different maximum-weight constraints.

3.4 Pseudocode for Performance Statistics

The final subproblem evaluates the realized performance of the strategy. This step is not used to choose the portfolio weights. It is only used after the weights have been computed, in order to compare the entropy-based portfolio with two reference portfolios: the equal-weight top-10 portfolio and the S&P 500 benchmark.

Algorithm 4 Performance comparison

Require: strategy weights w_t; active-universe returns r_t; S&P 500 returns R_t^{SP500}
Ensure: total return, annualized return, Sharpe ratio, and maximum drawdown for each portfolio

for each trading date t **do**
  $R_t^{strat} \leftarrow w_t^\top r_t$ ▷ entropy-based portfolio return
  $R_t^{EW} \leftarrow \frac{1}{10} \sum_{i \in \mathcal{U}_t} r_{i,t}$ ▷ equal-weight top-10 return
  store R_t^{strat}, R_t^{EW}, and R_t^{SP500}
end for
for each return series R_t in $\{R_t^{strat}, R_t^{EW}, R_t^{SP500}\}$ **do**
  $V_t \leftarrow \prod_{\tau \le t}(1 + R_\tau)$ ▷ cumulative wealth path
  $R^{tot} \leftarrow V_T - 1$ ▷ total return
  $R^{ann} \leftarrow V_T^{252/M} - 1$ ▷ annualized return over M trading days
  $\sigma^{ann} \leftarrow \sqrt{252} \cdot \text{std}(R_t)$ ▷ annualized volatility
  $\text{Sharpe} \leftarrow R^{ann} / \sigma^{ann}$
  $DD_t \leftarrow V_t / \max_{\tau \le t} V_\tau - 1$ ▷ drawdown path
  $\text{MaxDD} \leftarrow \min_t DD_t$
end for
return the performance statistics table

The comparison step summarizes the behavior of the three return series using the same metrics. Total return and annualized return measure growth, the Sharpe ratio measures return per unit of volatility, and maximum drawdown measures the worst peak-to-trough loss along the cumulative wealth path.

4 Toy examples

This section illustrates the algorithm on simplified inputs before moving to the full empirical simulation. The goal is not to reproduce the full backtest, but to show how the main subproblems behave: the computation of Shannon entropy, the interpretation of entropy values, and the translation from regime classification to portfolio weights.

4.1 Manual Shannon Entropy Calculation

The first example isolates the entropy calculation. Consider a short rolling window of six daily returns, written in percentage points, with $B = 8$ bins:

$$x = (-6.00\%, -2.00\%, -1.45\%, 0.00\%, +0.15\%, +2.00\%).$$

The entropy routine builds B equal-width bins on the window range $[\min(x), \max(x)]$. Here the range is $[-6\%, +2\%]$, so the bin width is

$$\Delta = \frac{2 - (-6)}{8} = 1\%.$$

Therefore, the bin edges are

$$-6, -5, -4, -3, -2, -1, 0, +1, +2.$$

Each return falls into one bin:

Return	Bin interval (%)	Bin number
-6.00%	$[-6, -5)$	1
-2.00%	$[-2, -1)$	5
0.00%	$[0, +1)$	7
$+0.15\%$	$[0, +1)$	7
$+2.00\%$	$[+1, +2]$	8

Table 1: Manual bin assignment for the first entropy toy window.

Only four bins are occupied, with counts $1, 2, 2, 1$. After dividing by the six observations, the non-zero probabilities are

$$p = \left(\frac{1}{6}, \frac{2}{6}, \frac{2}{6}, \frac{1}{6}\right).$$

The normalized Shannon entropy is therefore

$$H_{\text{norm}} = -\frac{1}{\log_2 8} \sum_b p_b \log_2(p_b) \approx 0.639.$$

This window is moderately disordered because the observations are spread across several magnitude bins.

Now consider a two-state window that only alternates between two values:

$$x' = (-2.00\%, +2.00\%, -2.00\%, +2.00\%, -2.00\%, +2.00\%).$$

Although the swings are large, only two bins are occupied. The probabilities are

$$p = \left(\frac{1}{2}, \frac{1}{2}\right),$$

so the normalized entropy is

$$H_{\text{norm}} = -\frac{1}{\log_2 8} \left[\frac{1}{2} \log_2 \left(\frac{1}{2}\right) + \frac{1}{2} \log_2 \left(\frac{1}{2}\right)\right] = \frac{1}{3} \approx 0.333.$$

The important point is that entropy is not a volatility measure. It does not increase just because returns are large in absolute value. It increases when the observations are spread across many bins.

4.2 Synthetic Shannon Entropy Example

The notebook also generates a larger toy example with 60 observations per regime. The three synthetic regimes are generated in Python with a fixed random seed, so that the example is fully reproducible.

The first series is the `structured_trend`. It is built as an almost linear path from -0.6% to $+0.6\%$, with a small uniform perturbation added to each observation:

$$r_i = -0.006 + 0.012\frac{i}{59} + \varepsilon_i, \qquad \varepsilon_i \sim U(-0.0003, 0.0003), \qquad i = 0, \ldots, 59.$$

The second series is the `two_state_regime`. Each observation can take only one of two values:

$$r_i \in \{-0.02, +0.02\}.$$

The third series is the `noisy_regime`. It is generated from a centered Gaussian distribution:

$$r_i \sim N(0, 0.012^2).$$

These three series are useful because they separate different notions of order. The two-state regime is structured because returns occupy only two possible levels. The noisy regime is more dispersed. The structured trend is smooth in time, but its observations are spread almost uniformly across the adaptive bins. This makes it useful for showing one limitation of histogram-based entropy.

The same entropy function used in the real backtest is applied without special treatment.

Example	Obs.	Entropy	Order score	Mean return
`structured_trend`	60	0.9978	0.0022	-0.000021
`two_state_regime`	60	0.3333	0.6667	0.000000
`noisy_regime`	60	0.8689	0.1311	-0.000314

Table 2: Toy entropy results generated by the final script and notebook.

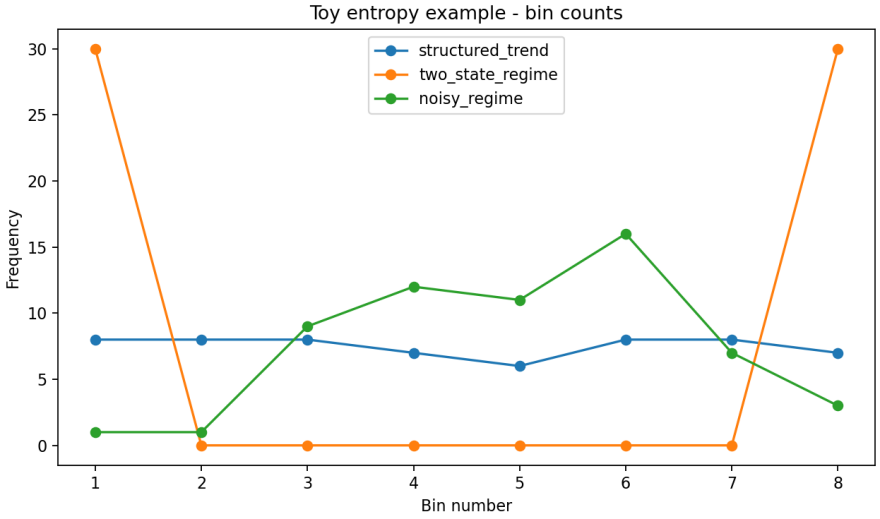

Figure 1: Toy entropy bin counts. The two-state regime has low entropy because observations occupy only two bins.

This toy example also shows one limitation of histogram-based entropy. The `structured_trend` series has very high entropy, even though it is not a noisy regime in the usual financial sense. This happens because the entropy calculation ignores the time order of returns and only observes how the values are distributed across bins. If a smooth and low-volatility trend fills the adaptive bins almost uniformly, the entropy can become high.

This is not a contradiction; it is a limitation of entropy as a measure of distributional disorder. Entropy is useful for detecting whether returns are concentrated in a few empirical states or spread across many states, but it does not directly measure trend, volatility, or expected return. In the full strategy, this risk is reduced by using a sufficiently long rolling window and by using entropy only to classify the regime. The expected-return input remains the plain rolling mean. In some cases, a very uniform and quiet distribution may also be interpreted as a warning signal, a possible "quiet before the storm", rather than as a direct buy or sell signal.

4.3 From Entropy to Weights: Portfolio Allocation

The entropy-only examples show how the signal is built. This mini-allocation example starts from the next stage of the algorithm: entropy, rolling quantiles, rolling means, and the covariance matrix are treated as already available inputs. The goal is only to show how a regime signal is translated into portfolio weights.

Consider a five-name universe

$$\mathcal{U} = \{A, B, C, D, E\}, \qquad K = 5.$$

The maximum-weight rule is scaled according to the number of active names:

$$c_{\text{Bullish}} = \frac{3}{K} = 60\%, \qquad c_{\text{Neutral}} = \frac{1.5}{K} = 30\%, \qquad c_{\text{Bearish}} = \frac{1}{K} = 20\%.$$

Thus, Bullish allows a stock to receive up to three times the equal-weight allocation, Neutral allows up to one and a half times equal weight, and Bearish collapses to the equal-weight book.

The rolling expected returns and covariance matrix come from the following short return history, written in percent. The last column reports the rolling mean used as expected-return input:

Stock	Daily returns (%)								$\widehat{\mu}_i = \bar{r}_i$
	1	2	3	4	5	6	7	8	
A	0.6	0.8	0.5	0.7	0.9	0.6	0.8	0.7	0.700%
B	2.0	-1.8	1.5	-2.2	0.3	1.9	-1.0	0.8	0.187%
C	1.2	-0.3	0.9	1.5	-0.6	1.1	0.4	0.8	0.625%
D	-0.2	-0.5	0.1	-0.4	-0.3	0.0	-0.6	0.2	-0.213%
E	0.3	0.4	0.2	0.5	0.4	0.3	0.4	0.3	0.350%

Table 3: Five-name toy allocation input and rolling-mean expected returns.

The trailing sample covariance matrix, multiplied by 10^4, is

$$\Sigma \times 10^4 = \begin{pmatrix} 0.017 & -0.130 & -0.074 & -0.024 & 0.009 \\ -0.130 & 2.770 & 0.315 & 0.360 & -0.131 \\ -0.074 & 0.315 & 0.548 & 0.080 & -0.013 \\ -0.024 & 0.360 & 0.080 & 0.084 & -0.021 \\ 0.009 & -0.131 & -0.013 & -0.021 & 0.009 \end{pmatrix}.$$

For this reduced example, the same expected-return vector and covariance matrix are used in both displayed states. This isolates the effect of the regime-dependent cap. In the full backtest, instead, $\widehat{\mu}_t$ and Σ_t are recomputed at every trading date from the current rolling windows.

Now suppose that the entropy and quantile subproblems have produced the following two states at two different dates:

State	$\bar{H}_t$	$q_{lo,t}$	$q_{hi,t}$	Phase
State 1	0.430	0.500	0.720	Bullish
State 2	0.580	0.470	0.690	Neutral

Table 4: Two regime states used in the mini-allocation example.

In State 1, average entropy is below the lower threshold, so the phase is Bullish and the cap is 60%. In State 2, average entropy lies between the two thresholds, so the phase is Neutral and the cap is 30%. Therefore, the two optimizations use the same return and covariance inputs, but they impose different maximum-weight constraints.

With $\lambda = 4$, the optimizer solves

$$\min_{w} \left(\frac{\lambda}{2} w^\top \Sigma w - \widehat{\mu}^\top w \right), \qquad \sum_{i \in \mathcal{U}} w_i = 1, \qquad 0 \le w_i \le c_t.$$

The resulting weights are:

Stock	Bullish cap 60%	Neutral cap 30%	Bearish equal weight
A	60.0%	30.0%	20.0%
B	0.0%	10.0%	20.0%
C	40.0%	30.0%	20.0%
D	0.0%	0.0%	20.0%
E	0.0%	30.0%	20.0%

Table 5: Toy portfolio weights under the two displayed states and the Bearish equal-weight reference.

The example shows how the same rolling inputs can lead to different allocations when the entropy regime changes. In the Bullish state, the optimizer is allowed to concentrate more aggressively: stock A reaches the 60% cap and stock C receives the remaining 40%. In the Neutral state, the 30% cap prevents that concentration, so the portfolio is spread across A, C, and E, with a smaller residual position in B. In a Bearish state, the optimizer would be skipped and the portfolio would be set directly to equal weight.

Entropy therefore selects the regime and the constraint level; it does not directly assign the individual weights. The weights are still determined by the mean-variance optimization problem under the cap imposed by the current phase.

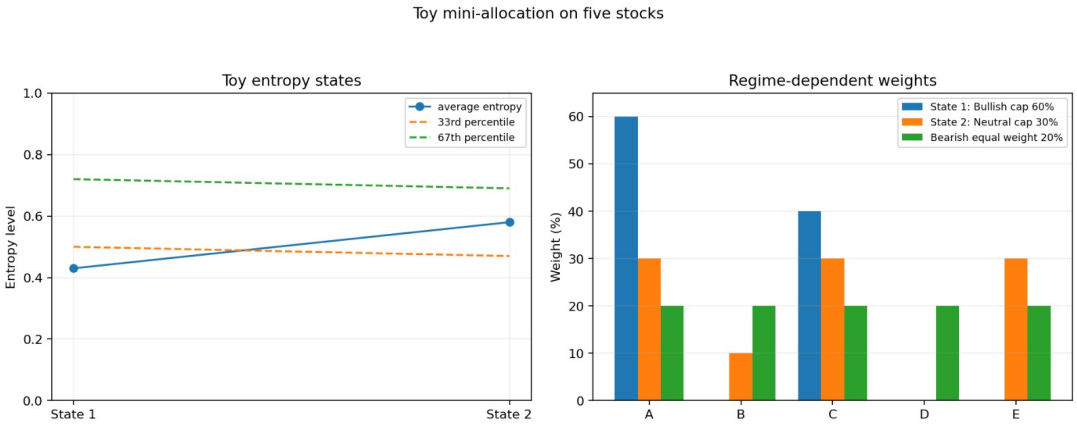

Figure 2: Toy mini-allocation with five stocks and two displayed states. The first state is Bullish and allows a 60% cap, while the second state is Neutral and restricts the cap to 30%. The Bearish reference is the 20% equal-weight book.

These toy examples should be read as isolated illustrations of the main algorithmic subproblems, not as a complete version of the final strategy. The first examples isolate the entropy calculation and show how Shannon entropy reacts to different empirical distributions of returns. The mini-allocation then starts from already available rolling inputs and focuses only on the translation from regime classification to portfolio weights. In the full empirical simulation, all these components are combined in a single daily pipeline: rolling statistics are updated through time, entropy thresholds are computed from past observations, the active top-10 universe changes by date, and the strategy is finally evaluated against the equal-weight portfolio and the S&P 500 benchmark.

5 Algorithm Analysis

This section contains the part with the highest algorithmic weight. It presents the Python implementation used in the simulations and in the toy examples, then discusses the data structures and the overall computational complexity. The previous section described the algorithmic subproblems at a high level; here the same logic is connected to the actual implementation objects, rolling updates, solver calls, output tables, and cost drivers.

5.1 Python Implementation Overview

The notebook is organized as a pipeline of functions and small classes. The prepared CSV files are loaded first. The price panel is converted into daily simple returns. The active-universe file is converted into a date-indexed dictionary of integer stock positions. Then the algorithm creates one rolling state per stock, one rolling quantile object for the active-universe entropy series, and three output tables: daily returns, stock-level weights, and diagnostics.

The same core functions are used in both the real simulation and the toy examples. The synthetic entropy example calls the same rolling entropy logic on artificial return windows. The mini-allocation example calls the same mean-variance solver on a reduced five-name universe. Therefore, the examples are not separate procedures: they are reduced-scale demonstrations of the same algorithmic components used in the final empirical run.

5.2 Prepared Inputs and Integer Indexing

The algorithm starts from prepared CSV files. These files were built from an Eikon dataset before the notebook execution. The raw Eikon structure is not used directly inside the algorithm. Instead, the data are reorganized into a simpler input format designed specifically for the daily portfolio procedure.

The three input files are:

1. `dataset.csv`: contains the date column, the S&P 500 benchmark, and the normalized stock price columns.
2. `top10_dictionary.csv`: contains, for each date, the current top-10 tickers and their corresponding stock-column indices.
3. `ticker_mapping.csv`: records the mapping between ticker names, stock-matrix indices, and dataset-column indices.

A simplified row of `dataset.csv` is:

```
Date,SP500,AAPL,AIG,AMZN,AVGO,...,VZ
2001-02-05,99.1552,93.3526,103.7285,83.3935,,...,95.5414
```

The empty field after `83.3935` represents a missing observation for that stock on that date. The parser keeps missing values as `None`; the rolling state later decides whether a stock window is ready.

The file `top10_dictionary.csv` stores the active universe directly. A simplified row is:

```
Date,Top 10 tickers,Top 10 stock indices
2001-01-31,"GE, MSFT, C, XOM, ..., IBM","9, 20, 6, 31, ..., 11"
```

The ticker field is useful for interpretation, while the index field is the one used by the algorithm. These indices are zero-based positions in the stock matrix after removing the `Date` column and the S&P 500 benchmark column.

The file `ticker_mapping.csv` makes this convention explicit. A simplified extract is:

```
Ticker,StockMatrixIndex,DatasetColumnIndex
AAPL,0,2
```

The column `StockMatrixIndex` is the integer position used by the algorithm after removing `Date` and `SP500`, while `DatasetColumnIndex` records the original column position in `dataset.csv`. This mapping separates interpretation from computation: ticker names remain available for reading the results, but the daily portfolio loop works directly with integer positions.

This indexing choice is important because the active universe changes every trading day. Once the top-10 file has been read, the algorithm can retrieve the active stocks as a list of ten column indices and use them directly in the return matrix. This avoids repeated ticker matching during the simulation and makes the rolling updates, entropy computation, optimization inputs, and realized-return calculation easier to implement.

5.3 Rolling Stock State Implementation

The algorithm maintains one rolling state for each stock. This is necessary because the active universe changes through time. A stock may enter the top-10 after being inactive for many dates, so its rolling history must already be available when it becomes active.

Each `RollingStockState` stores a fixed-size buffer, a valid-observation count, two running sums, and two monotonic queues:

1. `buffer`: a `deque` containing the most recent W return entries;
2. `count`: the number of valid returns inside the buffer;
3. `sum_x`: the sum of valid returns currently inside the buffer;
4. `sum_x2`: the sum of squared valid returns currently inside the buffer;
5. `min_q`: a monotonic queue for the rolling minimum;
6. `max_q`: a monotonic queue for the rolling maximum.

The role of the rolling state is to avoid recomputing everything from scratch at every date. The state stores both the raw rolling window and the aggregate information needed by the algorithm. The raw window is needed for entropy and covariance, while the running sums and monotonic queues allow some statistics to be read directly.

The monotonic queue is the data structure used to maintain the rolling minimum and maximum:

```
class MonotonicQueue:
    def __init__(self, mode):
        self.mode = mode
        self.q = deque()

    def push(self, time_index, value):
        if value is None:
            return
        if self.mode == "min":
            while len(self.q) > 0 and self.q[-1][1] > value:
                self.q.pop()
        else:
            while len(self.q) > 0 and self.q[-1][1] < value:
                self.q.pop()
        self.q.append((time_index, value))

    def expire(self, first_valid_index):
        while len(self.q) > 0 and self.q[0][0] < first_valid_index:
            self.q.popleft()

    def top(self):
        if len(self.q) == 0:
            return None
        return self.q[0][1]
```

The queue stores pairs (`time_index`, `return_value`). For the minimum queue, larger dominated values are removed from the back before inserting the new value. For the maximum queue, smaller dominated values are removed. Expired observations are removed from the front. Since every observation can enter and leave the deque at most once, the amortized update cost is $O(1)$.

The rolling stock state uses this queue together with the buffer and running sums:

```
class RollingStockState:
    def __init__(self, window):
        self.W = window
        self.buffer = deque()
        self.sum_x = 0.0
        self.sum_x2 = 0.0
        self.count = 0
        self.time_index = -1
        self.min_q = MonotonicQueue("min")
        self.max_q = MonotonicQueue("max")

    def push(self, value):
        self.time_index += 1

        if len(self.buffer) == self.W:
            old_value = self.buffer.popleft()
            if old_value is not None:
                self.sum_x -= old_value
                self.sum_x2 -= old_value * old_value
                self.count -= 1

        self.buffer.append(value)

        if value is not None:
            self.sum_x += value
            self.sum_x2 += value * value
            self.count += 1
            self.min_q.push(self.time_index, value)
            self.max_q.push(self.time_index, value)

        first_valid_index = self.time_index - self.W + 1
        self.min_q.expire(first_valid_index)
        self.max_q.expire(first_valid_index)

    def ready(self):
        return len(self.buffer) == self.W and self.count == self.W
```

The update has constant amortized cost for one stock. The deque operation is $O(1)$, the running-sum update is $O(1)$, and each monotonic-queue update is $O(1)$ amortized. Missing values are stored in the buffer but are not inserted into the sums or queues. A stock is usable only when `ready()` is true, meaning that the buffer contains exactly W valid returns.

5.4 Rolling Statistics and Their Costs

The rolling state supports several statistics with different computational costs. The first group is read directly from stored quantities:

Operation	Data used	Cost	Reason
Rolling mean	`sum_x`	$O(1)$	The sum is updated when the window moves.
Rolling variance	`sum_x`, `sum_x2`	$O(1)$	The squared sum is updated together with the ordinary sum.
Rolling minimum	`min_q`	$O(1)$	The current minimum is always at the front of the monotonic queue.
Rolling maximum	`max_q`	$O(1)$	The current maximum is always at the front of the monotonic queue.
Readiness check	`buffer`, `count`	$O(1)$	It only checks the window length and valid-observation count.

Table 6: Constant-time statistics provided by `RollingStockState`.

The second group requires scanning the rolling window. The main example is entropy. Even if the minimum and maximum are available in constant time, entropy must count how the W returns are distributed across the B adaptive bins:

$$\text{entropy cost per active stock} = O(W + B).$$

The $O(W)$ term comes from assigning each return to a bin. The $O(B)$ term comes from scanning the bin counts to compute the Shannon sum. Since the strategy computes entropy only for the active top-10 universe, the daily entropy cost is

$$O(K(W + B)).$$

The adaptive nature of the bins is the reason why the histogram is rebuilt. When the rolling minimum or maximum changes, the bin edges change as well, so old bin assignments cannot safely be reused.

5.5 Rolling Quantile Structure

The rolling quantile object is separate from the stock-level states. It stores the recent history of the active-universe average entropy $\bar{H}_t$. It must support two operations at the same time: removing the oldest entropy observation and reading the 33rd and 67th percentiles.

For this reason, the object stores the same values twice:

1. `buffer`: a deque storing entropy values in chronological order;
2. `sorted_values`: a sorted list storing the same values in increasing order.

The relevant implementation is:

```
class RollingQuantile:
    def __init__(self, window):
        self.W = window
        self.buffer = deque()
        self.sorted_values = []

    def push(self, value):
        if value is None:
            return
        if len(self.buffer) == self.W:
            old_value = self.buffer.popleft()
            position = bisect_left(self.sorted_values, old_value)
            self.sorted_values.pop(position)
        self.buffer.append(value)
        insort(self.sorted_values, value)

    def ready(self):
        return len(self.buffer) == self.W

    def quantile(self, q):
```

```
        if not self.ready():
            return None
        n = len(self.sorted_values)
        r = q * (n - 1)
        lo = int(math.floor(r))
        hi = int(math.ceil(r))
        if lo == hi:
            return self.sorted_values[lo]
        w = r - lo
        return (1.0 - w) * self.sorted_values[lo] + w * self.sorted_values[hi]
```

The deque update is $O(1)$. The call to `bisect_left` finds the position in $O(\log Q)$ time, but the removal from a Python list may shift elements, so the total removal cost is $O(Q)$. The insertion through `insort` also has a binary-search component and a shifting component, so its total cost is $O(Q)$. Therefore,

$$\text{rolling quantile update cost} = O(Q).$$

Once the sorted list is maintained, reading a quantile is $O(1)$ because the algorithm only reads two adjacent positions and applies linear interpolation. In this project, two quantiles are read daily, so the quantile-read cost remains $O(1)$.

5.6 Covariance Matrix and Optimization Layer

The covariance matrix cannot be obtained from the one-dimensional summaries of each stock. It needs aligned joint observations for all active names. Therefore, when the active universe is ready, the algorithm extracts the rolling buffers of the active stocks and builds a matrix of size $W \times K$.

The implementation uses the active buffers to construct a DataFrame and then calls the sample covariance routine:

```
def covariance_from_states(states, active_indices):
    data = {}
    for j in active_indices:
        data[str(j)] = states[j].values_as_list()
    return pd.DataFrame(data).cov().values
```

The construction of the active block costs $O(WK)$ because the algorithm copies W returns for each of the K active names. The covariance computation costs

$$O(WK^2),$$

because all pairwise covariances among the K active names are computed using W observations.

The optimization layer is used only in Bullish and Neutral phases. In Bearish phases, the optimizer is skipped and the portfolio is set directly to equal weight. In optimizing phases, the project calls SLSQP on the capped mean-variance problem:

$$\min_w \left(\frac{\lambda}{2} w^\top \Sigma_t w - \widehat{\mu}_t^\top w \right), \qquad \sum_i w_i = 1, \qquad 0 \le w_i \le c_t.$$

The core weight-construction logic is:

```
def solve_mean_variance_slsqp(mu, covariance, cap, risk_aversion):
    n = len(mu)

    lower_bounds = np.zeros(n)
    upper_bounds = np.full(n, cap)
    x0 = np.full(n, 1.0 / n)

    covariance = 0.5 * (covariance + covariance.T)
    covariance = covariance + np.eye(n) * 1e-8
```

```
    def objective(w):
        return (
            0.5 * risk_aversion * float(w @ covariance @ w)
            - float(mu @ w)
        )

    def gradient(w):
        return risk_aversion * (covariance @ w) - mu

    constraint = {
        "type": "eq",
        "fun": lambda w: np.sum(w) - 1.0,
        "jac": lambda w: np.ones_like(w),
    }

    result = minimize(
        objective,
        x0,
        method="SLSQP",
        jac=gradient,
        bounds=list(zip(lower_bounds, upper_bounds)),
        constraints=[constraint],
        options={"ftol": 1e-10, "maxiter": 120},
    )

    if result.success:
        return result.x, True, result.message

    return x0, False, result.message
```

This code shows the objects passed to the numerical optimizer. The vector x_0 is the equal-weight starting point. The lower and upper bounds impose the long-only and regime-cap constraints. The equality constraint enforces the budget condition $\sum_i w_i = 1$. The covariance matrix is symmetrized and regularized with a small diagonal term for numerical stability.

The objective evaluation contains the quadratic form $w^\top \Sigma_t w$. For a dense $K \times K$ covariance matrix, this evaluation costs $O(K^2)$. The analytical gradient

$$\nabla f(w) = \lambda \Sigma_t w - \widehat{\mu}_t$$

also costs $O(K^2)$, because it requires a dense matrix-vector product. The equality constraint and the bound checks are lower-order operations, with cost $O(K)$.

The solver itself is not reimplemented in the project. The project builds the rolling inputs and the regime-dependent constraints, then delegates the continuous constrained optimization to SLSQP. Since SLSQP solves a constrained quadratic problem through an iterative numerical procedure, we represent its cost with a dense-solver proxy. If I is the number of solver iterations, the optimization contribution is written as

$$O(IK^3).$$

This proxy accounts for the internal linear-algebra work required by the constrained solver across iterations. The exact internal implementation is part of SciPy and is not analyzed as custom project code.

From the project-code perspective, building $\widehat{\mu}_t$ from the active stock states costs $O(K)$, because each rolling mean is read in constant time. Building Σ_t costs $O(WK^2)$. In Bullish and Neutral regimes, the allocation layer is therefore summarized as

$$O(K) + O(WK^2) + O(IK^3).$$

In Bearish regimes, the covariance-and-solver layer is skipped and the portfolio is assigned directly as

$$w_i = \frac{1}{K}, \qquad i = 1, \ldots, K,$$

which costs only $O(K)$.

Since $K = 10$ in the real backtest, the optimization problem is small in absolute dimension. However, the solver term is still included in the worst-case analysis because it is part of the daily allocation pipeline whenever the phase is Bullish or Neutral.

5.7 Main Loop and Output Tables

The main simulation loop connects the rolling structures to the allocation step. At each valid trading date, the algorithm reads the active top-10 indices, computes the current active-universe entropy, compares it with the rolling quantile thresholds, assigns the market phase, and then builds the portfolio weights through either SLSQP or the Bearish equal-weight rule.

The update order is the main implementation detail. Entropy thresholds are read before the current average entropy is inserted into the rolling quantile structure. Similarly, date-t returns are pushed into the stock rolling states only after the date-t portfolio decision and realized return have been recorded. Therefore, the allocation at date t uses only information already available before that return enters the rolling windows.

The loop produces three output tables. The `daily` table stores one row per valid date with phase, entropy values, portfolio returns, benchmark returns, and solver status. The `weights` table stores one row per active stock and valid date, so its size is $O(TK)$. The `diagnostics` table stores one row per valid date with solver and feasibility information, so its size is $O(T)$, like `daily`.

5.8 Summary of Data Structures Used

The implementation does not rely on a single generic array structure. Different parts of the algorithm use different data structures according to the operation that must be performed. The notation used below is T for the number of trading days, N for the number of stock columns, $K = 10$ for the active universe size, $W = 126$ for the rolling stock window, $B = 8$ for the number of entropy bins, and $Q = 504$ for the rolling quantile window.

Object	Data structure	Main complexity	Purpose
dates, tickers	Python lists	access: $O(1)$	Preserve chronological order and stable column identity.
`prices, returns`	matrix / DataFrame	build returns: $O(TN)$	Store normalized prices and daily simple returns.
`active_by_date`	dictionary	lookup: $O(1)$ average; read active list: $O(K)$	Retrieve the active top-10 stock indices for each date.
`states`	list of `RollingStockState`	state access: $O(1)$	Store one independent rolling state for each stock.
`buffer`	deque	append/popleft: $O(1)$	Store the most recent W returns of one stock.
`sum_x, sum_x2`	scalar accumulators	update/read: $O(1)$	Compute rolling mean and variance without rescanning the window.
`min_q, max_q`	monotonic deques	update: $O(1)$ amortized; top: $O(1)$	Maintain rolling minimum and maximum for entropy bins.
`entropy counts`	list of length B	$O(W + B)$ per active stock	Count observations in adaptive bins and compute Shannon entropy.
`RQ.buffer`	deque	append/popleft: $O(1)$	Store average entropy values in time order.
`RQ.sorted_values`	sorted list	insert/remove: $O(Q)$; quantile read: $O(1)$	Maintain rank order for rolling entropy quantiles.
`active covariance block`	$W \times K$ matrix	build: $O(WK)$; covariance: $O(WK^2)$	Build the active covariance matrix.
`SLSQP input`	dense vectors/matrix	mean read: $O(K)$; solve proxy: $O(IK^3)$	Solve the capped mean-variance problem in optimizing regimes.
`output tables`	DataFrames	daily: $O(T)$; weights: $O(TK)$	Store returns, weights, phases, benchmarks, and diagnostics.

Table 7: Summary of the main data structures used by the implementation.

The most important design choice is the separation between stock-level rolling states and active-universe computations. The list `states` contains one `RollingStockState` for every stock column, not only for the stocks that are currently active. This is necessary because the top-10 universe changes through time: when a stock enters the active set, its rolling history must already be available.

The deque inside each stock state represents the rolling window as a FIFO object. The oldest return leaves from the front, and the newest return enters from the back. This makes the rolling update natural and avoids shifting all elements at every date. The scalar accumulators `sum_x` and `sum_x2` make rolling mean and variance available in constant time once the state is ready.

The two monotonic queues are used for the rolling minimum and maximum. Without them, the algorithm would have to scan the whole W-day buffer every time it needs the entropy range. With monotonic queues, each return can enter and leave the queue at most once. Therefore, their update cost is $O(1)$ amortized, and the current minimum or maximum is read directly from the front of the queue.

The rolling quantile object uses two structures because it needs two different orderings of the same entropy values. The deque preserves chronological order and identifies the oldest value when the window moves forward. The sorted list preserves rank order and allows the 33rd and 67th percentiles to be read by position. The price of this exact representation is that insertion and deletion in the sorted list cost $O(Q)$, because elements may have to be shifted.

5.9 Overall Computational Complexity

The full algorithm has an initial preparation cost and a daily online cost. Loading the prepared CSV files and computing daily simple returns require scanning the full $T \times N$ panel, so the preparation cost is

$$O(TN).$$

Building the active-universe dictionary from `top10_dictionary.csv` costs $O(TK)$, because each date stores $K = 10$ active stock indices. Since $K \ll N$, this term is smaller than the full return-matrix construction.

During the daily loop, the algorithm updates all N stock states, not only the active names. This is done so that every stock has an up-to-date rolling history when it enters the top-10 universe. Each stock update is $O(1)$ amortized, so the all-stock rolling update costs

$$O(N)$$

per trading day.

Once the active universe is read, entropy is computed only for the K active names. For one active stock, the rolling minimum and maximum are read in $O(1)$, but the entropy histogram must scan the W-day return buffer and then the B bin counts. Therefore, the active entropy layer costs

$$O(K(W + B))$$

per valid date.

The rolling quantile layer maintains the history of active-universe average entropy. Reading the two thresholds is $O(1)$, because the values are already sorted. Updating the sorted list, however, costs $O(Q)$, since insertion or deletion can shift elements in the list. Therefore, the rolling quantile contribution is

$$O(Q).$$

The covariance matrix is built only from the active names. The algorithm first extracts a $W \times K$ return block, which costs $O(WK)$. Then it computes all pairwise covariances among the K active names, which costs

$$O(WK^2).$$

In Bullish and Neutral regimes, the allocation layer calls SLSQP. Reading the expected-return vector costs $O(K)$, and the covariance matrix has already been built. If I denotes the number of SLSQP iterations, a standard dense-solver proxy for the optimization step is

$$O(IK^3).$$

In Bearish regimes, the optimizer is skipped and the portfolio is set directly to equal weight, which costs only $O(K)$.

Combining these terms, the daily cost in Bullish and Neutral regimes is

$$O\big(N + K(W + B) + Q + WK^2 + IK^3\big).$$

The terms correspond respectively to all-stock rolling updates, active entropy computation, rolling quantile maintenance, covariance construction, and numerical optimization.

In Bearish regimes, the covariance-and-solver layer is not needed for the allocation rule. The daily cost becomes

$$O\big(N + K(W + B) + Q + K\big).$$

Over the full backtest, the worst-case bound is obtained by assuming that the optimizer is called at every valid date:

$$O\left(T\left[N + K(W + B) + Q + WK^2 + IK^3\right]\right).$$

This is a conservative upper bound. In practice, Bearish dates are cheaper because they use the equal-weight allocation directly.

With the project values

$$K = 10, \qquad W = 126, \qquad B = 8, \qquad Q = 504,$$

the active-universe dimension is small. The main custom costs are therefore the entropy scans, the sorted-list updates for rolling quantiles, and the active covariance construction. The SLSQP cost is external to the custom algorithm, but it is included because it is part of the daily allocation pipeline in Bullish and Neutral regimes.

5.10 Memory Complexity

The notebook version stores the full prepared price and return matrices, so the loaded data require

$$O(TN)$$

memory. The online rolling stock states require

$$O(NW)$$

memory for the return buffers and monotonic-queue candidates. The rolling quantile structure requires

$$O(Q)$$

memory. At each valid allocation date, the active covariance block requires $O(WK)$ temporary memory and the covariance matrix requires $O(K^2)$ temporary memory.

The output tables require $O(T)$ memory for the daily and diagnostic tables and $O(TK)$ memory for the long weight table. Therefore, the total memory footprint of the notebook implementation is

$$O(TN + NW + Q + TK).$$

The $O(WK)$ and $O(K^2)$ covariance objects are temporary and smaller than the main stored matrices for the project dimensions.

This is an intentional tradeoff. A fully streaming implementation could avoid storing the full price and return matrices, but the Colab deliverable keeps them in memory because they make the computation reproducible, inspectable, and easy to audit.

6 Simulations

This section reports the execution of the algorithm on the real-scale dataset. At this stage, the goal is no longer to explain the individual components, but to show their combined effect on the full daily simulation: entropy filtering, regime classification, portfolio construction, benchmark comparison, and crisis-window behavior.

The results are presented in the execution order used for interpretation. First, we inspect the market filter produced by the entropy signal and its regime transitions. Then, we look at the portfolio weights generated by the regime-dependent constraints. Finally, we evaluate the realized performance against the benchmarks and isolate the behavior during major crisis periods.

6.1 Experimental Setup

The final simulation uses the prepared files `dataset.csv` and `top10_dictionary.csv`. The dataset ends on 2024-12-31. The portfolio series starts on 2003-08-08, which is the first trading date for which both the stock-level rolling windows and the rolling entropy quantile window are available.

At each valid trading date, the algorithm reads the current top-10 universe, computes the active-universe entropy signal, assigns the market phase, builds the portfolio weights, and records the realized returns of the entropy strategy and of the two benchmarks.

The main simulation parameters are:

1. stock-level rolling window: $W = 126$ trading days;
2. entropy bins: $B = 8$;
3. active universe size: $K = 10$;
4. entropy quantile window: $Q = 504$ trading days;
5. risk aversion: $\lambda = 4$;
6. minimum weight: 0;
7. maximum per-name weight: 30% in Bullish, 15% in Neutral, and 10% in Bearish;
8. optimizer: SLSQP in Bullish and Neutral regimes; equal weight in Bearish;
9. benchmarks: dynamic equal-weight top-10 portfolio and S&P 500 benchmark.

The implementation avoids look-ahead bias by using only information available before the daily realized return is inserted into the rolling states.

6.2 Entropy Market Filter and Regime Transitions

The table below reports the empirical distribution of the three regimes produced by the simulation. The classification is well balanced across the sample: Bullish covers about 39% of the valid days, while Neutral and Bearish cover about 30% each. The average entropy increases from Bullish to Bearish, confirming that the implemented filter behaves consistently with the regime definition.

Phase	Days	Share	Avg. entropy	Mean return
Bullish	2084	0.3872	0.7076	0.000693
Neutral	1634	0.3036	0.7519	0.000750
Bearish	1664	0.3092	0.7800	0.000342

Table 8: Market phase summary.

The diagnostic plot shows the time evolution of the market filter: the rolling top-10 return, the active-universe average entropy, the rolling entropy thresholds, and the resulting daily phase.

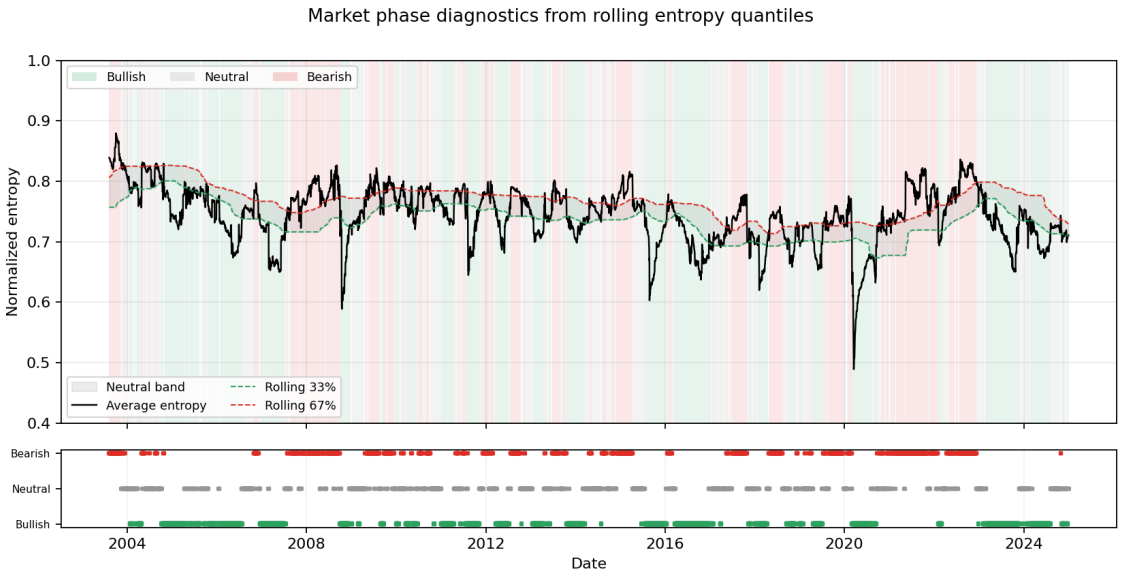

Figure 3: Market-phase diagnostics: 126-day rolling top-10 return, average entropy with rolling quantile bands, and daily Bullish/Neutral/Bearish label.

The transition matrix summarizes how persistent the entropy regimes are from one trading day to the next. The diagonal dominance indicates that the regimes tend to persist rather than switching randomly every day.

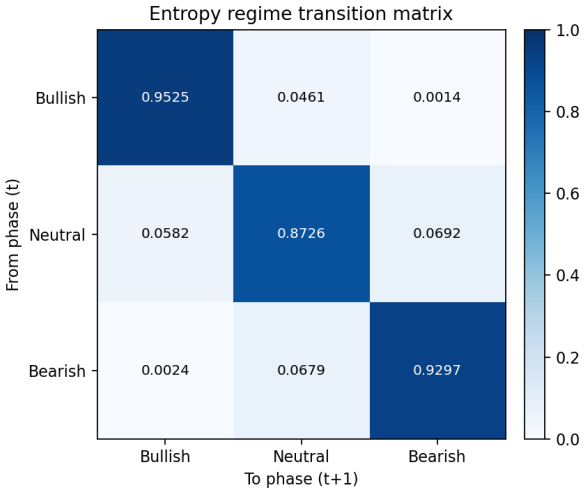

Figure 4: Empirical transition matrix between entropy regimes.

6.3 Portfolio Weights

After the market phase is assigned, the algorithm produces the daily portfolio allocation over the active top-10 universe. The heatmap below shows the realized weights through time. Zero-weight regions correspond either to inactive names or to active names that receive no allocation from the optimizer.

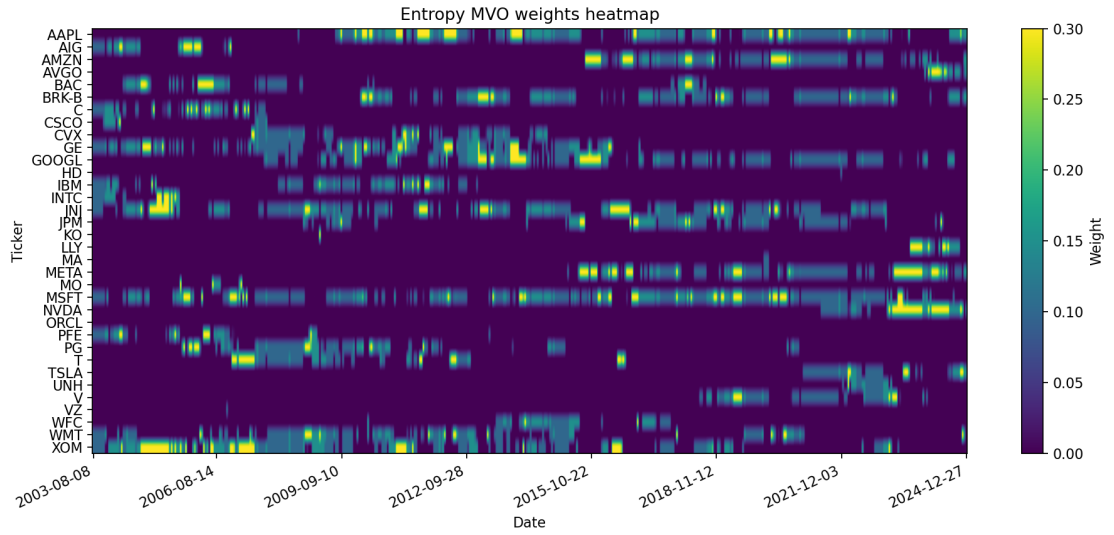

Figure 5: Entropy MVO portfolio weights through time. Inactive names and non-selected active names receive zero weight.

6.4 Portfolio Performance Against Benchmarks

The final performance comparison evaluates the entropy market-phase portfolio against the dynamic equal-weight top-10 benchmark and the S&P 500 benchmark. All wealth series are rebased to 1 at the first optimized date and compounded through time.

Strategy	Obs.	Ann. return	Ann. vol.	Sharpe	Max DD	Final wealth
Entropy market-phase MVO	5382	0.1405	0.2008	0.6995	-0.4899	16.5608
Equal-weight top-10 benchmark	5382	0.1150	0.1940	0.5926	-0.5154	10.2185
S&P 500 benchmark	5382	0.1098	0.1876	0.5851	-0.5480	9.2466

Table 9: Full-sample performance metrics on the provided CSV data.

Over the full sample, the entropy market-phase MVO portfolio obtains the highest annualized return, Sharpe ratio, and final wealth. Its maximum drawdown is also smaller than the drawdowns of both benchmarks, although the strategy still remains exposed to large equity-market losses.

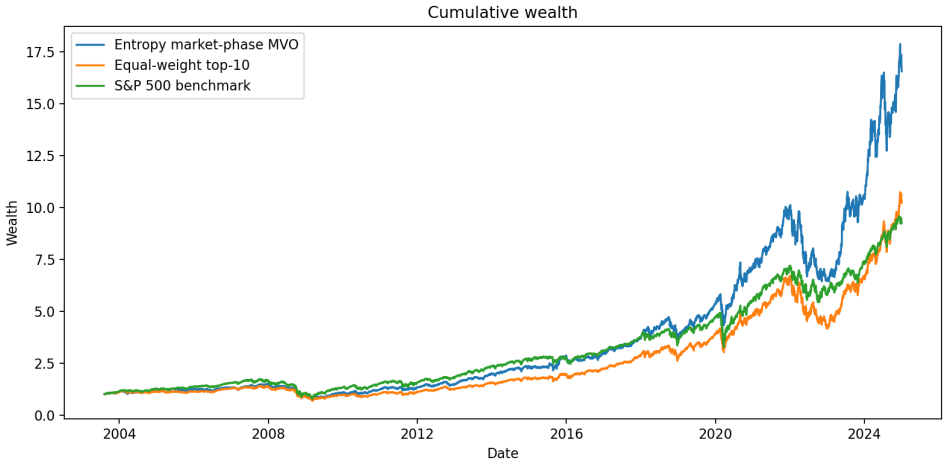

Figure 6: Cumulative wealth of the entropy market-phase portfolio, equal-weight top-10 benchmark, and S&P 500 benchmark.

6.5 Behavior During Crisis Windows

The final check isolates four stressed subperiods: the global financial crisis, the 2011 Euro debt and U.S. downgrade episode, the COVID crash, and the 2022 inflation-and-rates bear market. Each panel rebases wealth to 1 at the beginning of the corresponding window, so the comparison focuses on local crisis behavior rather than on full-sample compounding.

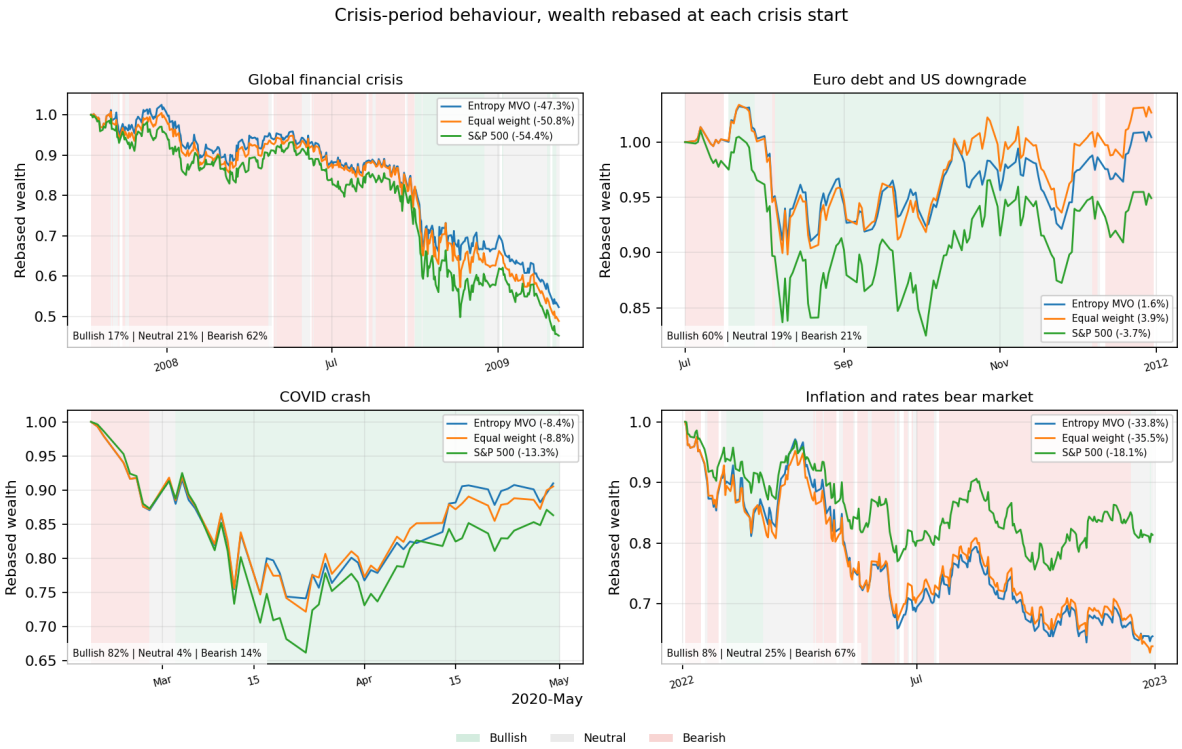

Figure 7: Crisis-period behavior of the entropy-regime mean-variance portfolio versus the dynamic equal-weight top-10 benchmark and the S&P 500 benchmark. Wealth is rebased to 1 at the start of each crisis window.

7 Conclusions

This project set out to solve a practical problem: building a daily long-only portfolio over a moving top-10 universe. The mechanism is a transparent, look-ahead-free rolling Shannon entropy that places the market on an ordered-to-disordered spectrum. Entropy is used only as the regime filter; the expected-return input is the plain rolling mean. Around that one number the rest of the rule is intentionally minimal: use prepared price-index inputs, keep inactive tickers at zero, restrict trading to the current top-10 by date, and set the per-name weight cap directly from the entropy regime.

The empirical result is positive but not one-dimensional. The entropy market-phase MVO has the highest full-sample annualized return, Sharpe ratio, final wealth, and the least severe maximum drawdown among the reported strategies. The equal-weight top-10 benchmark remains a strong and important baseline, and during crisis windows the entropy strategy often moves close to it because the Bearish gear intentionally diversifies. This is expected behavior, not a failure of the model. Entropy is a gear selector, not a return predictor.

The algorithm should therefore be interpreted as a disciplined portfolio-construction method, not as a return forecaster. The rolling mean is a deliberately weak tie-breaker, while the real decision, how aggressively to act, is carried by the entropy regime through the cap and the diversify-versus-concentrate switch. Entropy is kept directionless and scale-free so that it measures market disorder and nothing else, leaving direction to the rolling mean and risk to the covariance.

Potential improvements include adaptive binning, transaction-cost modeling, turnover constraints, and alternative entropy definitions such as permutation entropy.

Future Development: Discrete Weights and Graph Search

A natural future development is to replace or complement continuous SLSQP with a discrete allocation layer. In the current model, SLSQP chooses real-valued weights such as $w_i = 0.298657$. A discrete extension would force weights to a fixed grid, for example

$$w_i = \frac{x_i}{100}, \qquad x_i \in \mathbb{Z}, \qquad \sum_i x_i = 100.$$

This would make the strategy more implementation-oriented because real portfolios often require rounded weights, minimum lots, or percentage grids. It would also change the mathematical problem: the original problem is continuous constrained optimization, while the grid version is integer optimization.

A*, Dijkstra, and Bellman-Ford cannot directly replace SLSQP. They are graph algorithms and require finite nodes, edges, and path costs. The mean-variance objective contains the quadratic term $w^\top \Sigma w$, so the cost of assigning one weight depends on the other weights through covariance. A graph formulation is possible only after defining states, transitions, and a valid cost structure. For A*, one would also need an admissible heuristic, which is not trivial because of covariance interactions.

Exact grid enumeration is finite but not practical. With ten active stocks and a 1% grid, the number of feasible Bullish portfolios under a 30% cap is about 2.5 trillion, and the Neutral case under a 15% cap is still about 7.1 billion. A practical intermediate approach is a local 1% greedy search: start from equal weight, repeatedly test all feasible transfers of 1% from stock i to stock j, accept the best improving move, and stop when no one-step transfer improves the same mean-variance objective. This gives a local discrete optimum, not a certified global optimum.

The future-development hierarchy is therefore:

$$\begin{aligned} \text{SLSQP} + \text{rounding} &\to \text{independent } 1\% \text{ local search} \\ &\to \text{full graph/A*/branch-and-bound/integer quadratic solver.} \end{aligned}$$

The last level is the rigorous discrete optimization path, but it requires substantially more modeling work. The correct claim is not that A* is an automatic replacement for SLSQP; the correct claim is that a discrete graph or integer formulation could become a separate allocation layer for studying rounded weights and alternative algorithmic guarantees.

References

[1] C. E. Shannon. A Mathematical Theory of Communication. *Bell System Technical Journal*, 1948.

[2] H. Markowitz. Portfolio Selection. *The Journal of Finance*, 1952.

[3] G. C. Philippatos and C. J. Wilson. Entropy, Market Risk, and the Selection of Efficient Portfolios. *Applied Economics*, 1972.

[4] R. Zhou, R. Cai, and G. Tong. Applications of Entropy in Finance: A Review. *Entropy*, 2013.

[5] J. D. Hamilton. A New Approach to the Economic Analysis of Nonstationary Time Series and the Business Cycle. *Econometrica*, 1989.

[6] R. Jagannathan and T. Ma. Risk Reduction in Large Portfolios: Why Imposing the Wrong Constraints Helps. *The Journal of Finance*, 2003.

[7] J. Nocedal and S. Wright. *Numerical Optimization*. Springer, 2006.